ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

HONORIS UNITED UNIVERSITIES

Compte rendu de travaux pratiques
TP complémentaire Orange et KNIME

Ingénierie Informatique et Réseaux

Intelligence Artificielle & Sciences des Données

Sous le thème

Construire un workflow de Machine Learning visuel

Réalisé par :

Ismail Lahlou

Filière : AI | Groupe : G6

Module :

Déploiement d'IA

Année universitaire 2026-2027

1. Objectif et état de validation

Ismail Lahlou - AI - G6

Ce travail étudie un workflow de détection de fraude avec Orange et prépare son équivalent KNIME. Il complète le TP04 AutoML. Le document présente les calculs exécutés, leurs limites et les étapes nécessaires avant un rendu complet dans les deux applications.

Base technique adaptée du dépôt public ZED-abd/TP_Complementaire_Orange_KNIME. Les scripts de base et leur historique sont attribués à cette source. Les résultats marqués comme locaux ont été recalculés dans cette copie ; les illustrations Python ne sont pas des captures d'interface.

Élément	État
Données	Recalcul local : 5 000 transactions, 568 fraudes ; graine 7
Orange API	Exécution locale Orange 3.40.0 réussie ; métriques et prédictions vérifiées
Workflow Orange	14 widgets, 16 liens ; recharge et exécution locales réussies ; captures GUI à compléter
KNIME natif	Non exécuté ; aucun export .knwf ni capture KNIME disponible
Simulation préparatoire	Recalcul scikit-learn ; résultats distincts des mesures KNIME
Rapport	Compte rendu de travail ; compléter les captures et le parcours KNIME avant remise

Les caractéristiques sont montant, anciennete, freq_24h et nuit. La cible fraude doit être catégorielle : 0 ou 1. Avec seulement 11,36 % de fraudes, une exactitude élevée peut masquer un rappel insuffisant pour la classe 1.

2. Orange avant et après amélioration

Les scores portent sur la classe 1. Le Data Sampler Orange stratifié et déterministe utilise 3 750 lignes pour apprendre et 1 250 pour tester, dont 142 fraudes. Les paramètres des scripts sont conservés ; aucune valeur attendue ne remplace une mesure.

Avant

Modèle	AUC	F1 fraude	Précision	Rappel
Logistic Regression	0.9794	0.7810	0.8106	0.7535
kNN	0.8252	0.4783	0.6250	0.3873
Tree	0.8451	0.7559	0.8571	0.6761
Random Forest	0.9766	0.8129	0.8309	0.7958
Gradient Boosting	0.9891	0.8370	0.8828	0.7958

Après

Modèle	AUC	F1 fraude	Précision	Rappel
Logistic Regression	0.9794	0.7810	0.8106	0.7535
kNN	0.9641	0.7798	0.8000	0.7606
Tree	0.8451	0.7559	0.8571	0.6761
Random Forest	0.9840	0.8333	0.8582	0.8099
Gradient Boosting	0.9891	0.8370	0.8828	0.7958

La standardisation du kNN est apprise sur l’entraînement ; elle évite que le montant domine les distances. La forêt passe de 10 à 100 arbres. Les autres modèles restent identiques. Les scores décrivent ce protocole, sans démontrer une supériorité universelle.

3. Erreurs du Gradient Boosting

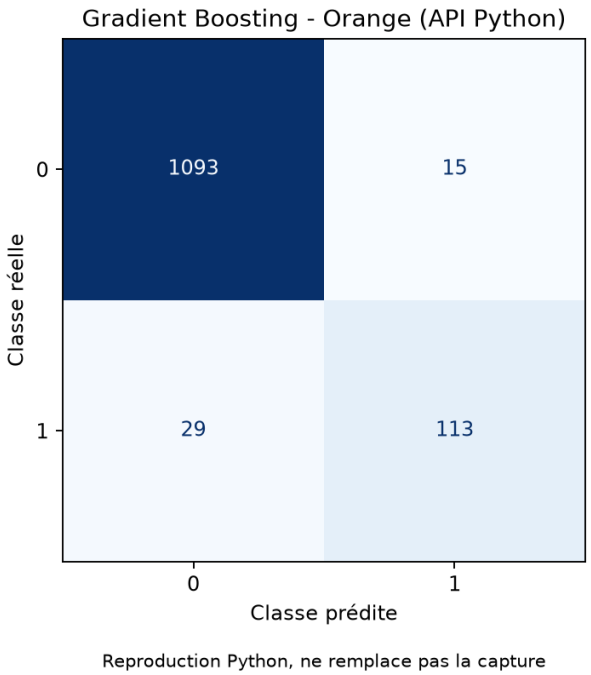
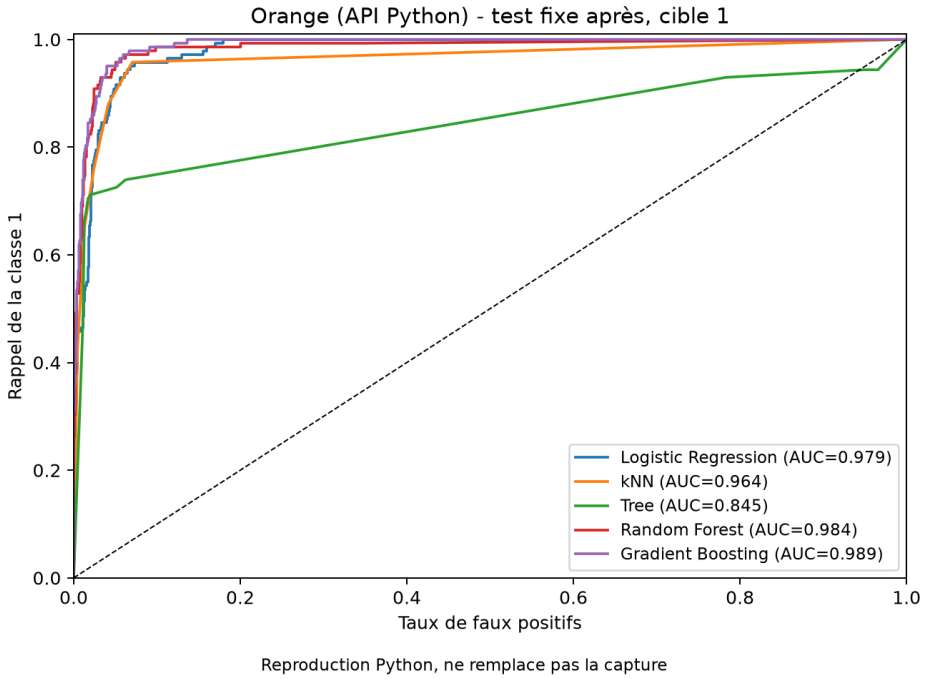


Figure calculée par Python depuis les résultats Orange ; ce n'est pas une capture du widget Confusion Matrix.

Matrice observée : [[1093, 15], [29, 113]]. Les lignes correspondent aux classes réelles et les colonnes aux prédictions. Les fausses alertes et les fraudes manquées ont des coûts métier distincts ; le F1 les résume sans exprimer ces coûts.

Le modèle Orange est enregistré dans models/fraude-gb.pkcls. Le script vérifie que le rechargement restitue les mêmes prédictions. Ce fichier dépend d'Orange et de ses bibliothèques ; son export seul ne constitue pas un service de production sécurisé.

4. Courbes ROC et validation croisée



ROC après amélioration, reproduction Python des probabilités calculées. Les captures Orange restent à réaliser.

Modèle	F1 test fixe	F1 CV 5 plis
Logistic Regression	0.7810	0.7438
kNN	0.7798	0.7803
Tree	0.7559	0.7035
Random Forest	0.8333	0.8193
Gradient Boosting	0.8370	0.8180

La CV prédit chaque observation hors de son pli d’apprentissage ; son protocole diffère du test fixe. La courbe ROC mesure le compromis rappel/taux de fausses alertes selon le seuil ; le seuil métier doit être choisi sur validation puis évalué sur un test indépendant.

5. KNIME et lien avec TP04

Le guide fourni décrit CSV Reader, choix de la cible nominale, Partitioning stratifié, puis trois branches Learner / Predictor / Scorer. Les résultats scikit-learn présents dans le dépôt servent seulement de préparation. Aucun score de simulation n’est présenté comme une mesure native KNIME.

Simulation scikit-learn	F1 classe 1
Logistic Regression	0.7016
Random Forest	0.7923
Gradient Boosting	0.8134

Référence personnelle réellement obtenue au TP04 : FLAML 60 s, lgbm, F1 0.8409. Nouvelle recherche de cette copie : xgboost, F1 0.8582. Les durées sous budget dépendent de la machine.

Orange utilise un test stratifié contenant 142 fraudes ; FLAML conserve le découpage externe du TP04, contenant 134 fraudes. Les scores ne constituent donc pas une comparaison à test identique. Les temps humains de manipulation ne sont pas mesurés et ne sont pas inventés.

Décision pédagogique

Orange facilite l’apprentissage et l’exploration. KNIME rend explicites les étapes du processus, à vérifier dans l’application. FLAML facilite la recherche automatisée et l’intégration Python. Le choix repose sur les utilisateurs, la reproductibilité et l’exploitation, au-delà du F1. Une mise en production exige contrat d’entrée, dépendances figées, tests, registre et suivi de dérive.

Réponses aux questions 1 et 2

Q1 Exactitude et F1

Nos mesures Orange (API Python) donnent au kNN une exactitude de 0.904, mais un F1 fraude de 0.478261. Cette différence vient du déséquilibre : la classe 0 représente 0.8864 des transactions. Une prédiction toujours égale à zéro atteint donc cette exactitude, avec un rappel et un F1 fraude nuls. Le kNN brut ne retrouve que 0.387324 des fraudes du test. L'exactitude masque ainsi une grande partie des erreurs qui intéressent la banque. Le kNN compare des distances ; montant, dont la médiane mesurée vaut 476.55, possède une échelle beaucoup plus grande que nuit, binaire, et peut dominer anciennete ou freq_24h. Les voisinages reflètent alors surtout le montant. La standardisation apprise uniquement sur l'apprentissage rééquilibre les attributs continus. Notre F1 passe à 0.779783 et le rappel à 0.760563. Cette amélioration observée justifie le prétraitement, sans garantir sa pertinence pour toutes les distributions futures. Les arbres reposent davantage sur des seuils que sur des distances et sont donc moins sensibles aux unités.

Q2 Reproductibilité

Fixer les graines permet d'attribuer une différence de score à une modification identifiable. Notre forêt Orange (API Python) obtient un F1 de 0.812950 avec dix arbres puis 0.833333 avec cent arbres, sur exactement le même test. Le code installé montre que Replicable training transmet random_state=0. Sans graine, les tirages de lignes et d'attributs changeraient : on pourrait retenir un modèle simplement favorisé par le hasard. Nous n'avons pas mesuré ici une forêt non déterministe ; les variations citées dans l'énoncé restent des valeurs attendues non reproduites. En MLOps, cette variabilité complique audit, débogage, comparaison et contrôles de non régression en intégration continue. Dans KNIME, il faut cocher Use static random seed ; dans scikit-learn, renseigner random_state ; dans FLAML, seed. Nous utilisons aussi la graine 42 pour les partages et la validation croisée Orange. Une graine ne suffit toutefois pas à reproduire des versions différentes : dépendances, données, paramètres, parallélisme et budget temps doivent également être consignés.

Réponses aux questions 3 et 4

Q3 Coût des erreurs et seuil

La matrice mesurée avec Orange (API Python) contient 29 fraudes manquées et 15 fausses alertes, pour 113 fraudes détectées. Une fraude manquée expose généralement la banque à une perte financière ; une fausse alerte provoque de la friction client et mobilise des analystes. Nous ne disposons pas de coûts métier chiffrés : affirmer un coût total serait inventer une information. Le volume d'alertes peut aussi saturer le traitement et rendre certaines fraudes moins visibles. Abaisser le seuil de décision favorise le rappel. Nous avons réellement recalculé les prédictions GB avec un seuil de 0,3 au lieu de 0,5 : la matrice devient [[1078,30],[16,126]], le rappel atteint 0.887324 et le F1 0.845638. Nous récupérons donc 13 fraudes supplémentaires, mais ajoutons 15 fausses alertes. Ce seuil illustre le compromis et n'est pas une recommandation de production. Il faut choisir le seuil sur validation selon les coûts et la capacité des équipes, puis vérifier sur un test indépendant. Les probabilités, les pondérations de classes et ROC Analysis avec coûts FP/FN aident cette décision.

Q4 Déclarer la cible

Dans le workflow KNIME décrit par l'énoncé, fraude est lue comme un entier par CSV Reader. Les Learners de classification demandent une cible nominale : Number to String convertit uniquement cette colonne en catégories textuelles « 0 » et « 1 ». Une cible numérique peut être absente de Target column ou empêcher la configuration du nœud. Convertir tous les attributs serait une erreur différente, car montant et les compteurs perdraient leur représentation quantitative. Dans Orange, le geste équivalent consiste à choisir Role=target et Type=categorical dans File, puis Apply. Notre domaine Orange (API Python) comporte bien quatre attributs, dont nuit catégorielle, et une cible fraude discrète ; le partage testé contient 568 fraudes au total. Si l'on conserve une cible continue, les widgets compatibles sélectionnent la régression et Test and Score propose notamment MSE/RMSE ; certains Learners exclusivement classificatoires peuvent refuser les données. Le contrôle du type et du rôle doit donc précéder toute lecture de F1. Cette vérification a été répétée lors de la recharge du workflow généré.

Réponses aux questions 5 et 6

Q5 Deux environnements visuels

Orange recalcule les widgets lorsque les données ou les paramètres changent. Dans notre reproduction Orange (API Python), Test and Score entraîne les cinq Learners sur 3750 lignes et évalue leurs prédictions sur 1250 autres. Pour un débutant, cette interaction facilite l'exploration : le passage du F1 kNN de 0.478 à 0.780 devient immédiatement interprétable. Elle peut toutefois masquer les entraînements déclenchés et rendre le protocole difficile à suivre sans annotations. Selon le fonctionnement décrit dans l'énoncé, KNIME demande une exécution explicite ; les feux signalent configuration, préparation et exécution. Learner et Predictor sont séparés, ce qui rend visibles le modèle appris et les données auxquelles il s'applique. Le workflow est plus long à construire, mais cette explicitation aide à organiser les contrôles et la maintenance en équipe. Les possibilités de partage via Hub et de déploiement présenté dans le bonus favorisent un processus plus structuré, sans assurer seules sa gouvernance. Nous n'avons pas manipulé l'interface KNIME : cette comparaison d'usage doit être complétée par les observations personnelles de l'étudiant.

Q6 Outils visuels et mise en production

Notre Gradient Boosting atteint 0.837037 de F1 avec Orange (API Python). La référence FLAML réellement exécutée en Python, de provenance simulation scikit-learn, retient xgboost avec un F1 externe de 0.858238. Les tests diffèrent : ces valeurs n'établissent pas une supériorité universelle. La valeur TP2 de 0,8669 reste une valeur attendue de l'énoncé, non reproduite. Les outils visuels gardent un intérêt pour apprendre, expliquer et prototyper avec des utilisateurs peu familiers du code. Orange sert l'exploration rapide ; KNIME explicite les étapes d'un processus ; FLAML et le code facilitent l'automatisation, les tests et l'intégration MLOps. Pour produire fraude-gb.pkcls, nous avons sauvegardé un vrai modèle et vérifié sa recharge. Cela reste insuffisant pour la production du TP05 : il faut figer les dépendances, notamment Orange, conserver le prétraitement dans le pipeline, définir un contrat de données, exposer une API de serving, ajouter tests et conteneurisation, enregistrer les versions dans un registre tel que MLflow, puis surveiller erreurs et dérive. Le seuil, sa justification métier et sa procédure de révision doivent aussi être gérés.

Compléments nécessaires et sources

Avant de présenter ce document comme un TP terminé : ouvrir le workflow Orange, vérifier les réglages et réaliser ses trois captures ; exécuter les trois branches KNIME, relever les statistiques de classe 1, exporter un vrai .knwf puis réaliser les deux captures KNIME. Les guides détaillés sont inclus dans reports/.

Les résultats et textes adaptés proviennent des scripts du dépôt de référence, conservés avec attribution. Le document personnel porte uniquement le nom d'Ismail Lahlou ; il ne revendique pas l'écriture initiale de tous les scripts ni une manipulation KNIME inexistante.

https://github.com/ZED-abd/TP_Complementaire_Orange_KNIME

https://github.com/lahlouabdeslam6-rgb/TP_Complementaire_Orange_KNIME

Source pédagogique : TP_Complementaire_Orange_KNIME.pdf inclus dans le dépôt. Les valeurs observées, journaux et verrous de versions sont archivés dans reports/.